

Материалы к экзамену ССФ — полностью



Все семь уроков одной страницей. Для чтения подряд и для сохранения в PDF: печать из браузера даёт готовый файл.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Accept all

Reject non-essential

Manage preferences



1. Из чего собран Cozystack

Три слоя, четыре варианта установки и почему платформа — это просто ещё один набор объектов Kubernetes.

Начнём с вопроса, который на экзамене задают чаще любого другого: **что такое Cozystack по своей природе?** Не «для чего он», а именно из чего сделан.

Ответ короткий: это Kubernetes, которому добавили новые типы объектов. Не форк, не надстройка над чужим API, не отдельный сервер управления. Когда вы просите платформу завести базу данных, вы создаёте объект — такой же, как под или сервис, только называется он Postgres. Дальше за ним следит программа-оператор и делает всё остальное.

Это стоит уложить в голове до всего прочего, потому что из этого следует почти всё остальное. Работает `kubectl`? Работает. Работает Terraform? Работает. Права раздаются обычным RBAC? Да, обычным.

Три слоя

Платформа собрана из трёх этажей, и путать их порядок — самая частая ошибка на экзамене.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



Cozystack

тенанты, базы, виртуалки, S3 — как объекты Kubernetes

Kubernetes

управляющий кластер: API, планировщик, операторы

Talos Linux

операционная система на каждом узле

Снизу вверх: Talos → Kubernetes → Cozystack. Каждый слой стоит на предыдущем.

Talos Linux — операционная система узлов (node OS). Её особенность в том, что у неё нет командной строки: ни SSH, ни консоли, ни пакетного менеджера. Настраивается она декларативно, через свой API: вы отправляете описание желаемого состояния, узел приводит себя к нему. Корневая файловая система при этом только для чтения.

Для администратора, привыкшего зайти на сервер и что-нибудь поправить, это поначалу неуютно. Смысл в другом: если зайти и поправить нельзя, то и состояние узла не расползается со временем. Два узла, собранные по одному описанию, останутся одинаковыми через год.

Kubernetes — управляющий кластер, который работает поверх Talos. Обычный, не переделанный: тот же API, те же контроллеры.

Cozystack — набор компонентов, устанавливаемых в этот кластер. Именно они добавляют новые типы объектов и следят за ними.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

1. **Talos на узлах.** Машины загружаются с образа платформы и получают машинное описание.
2. **Kubernetes.** Из этих узлов собирается управляющий кластер.
3. **Cozystack.** В кластер ставится сама платформа, дальше она разворачивает свои компоненты сама.



Третий этап устроен любопытно: платформа не устанавливает компоненты по списку, а описывает желаемый состав и передаёт его **FluxCD**. Дальше Flux сам приводит кластер к описанию и продолжает следить, чтобы состав не расходился. Обновление платформы — это смена версии в описании, а не пересборка руками.

Каждый компонент платформы Flux ставит отдельным **релизом** — объект называется `HelmRelease`, в командах сокращается до `hr`. Поэтому вопрос «встала ли платформа» — это вопрос о релизах, а не о подах:

```
kubectl get hr -A
```

Все в состоянии `READY True` — установка завершена.

Четыре варианта

Платформу ставят не целиком, а одним из четырёх наборов — под то, для чего она нужна. Названия стоит запомнить дословно, их спрашивают.

Вариант

Что даёт

`full`

всё: виртуализация, хранилище, сети, управляемые сервисы

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Из чего она состоит



Компонентов много, но на экзамене спрашивают, **кто за что отвечает**, а не полный список. Держите в голове такую карту:

Задача	Компонент
Виртуальные машины	KubeVirt
Дисковое хранилище	LINSTOR / DRBD
Объектное хранилище (S3)	SeaweedFS
Сеть подов	Cilium
Сеть тенантов, VPC	Kube-OVN
Внешние адреса	MetalLB
Метрики	VictoriaMetrics
Журналы	VictoriaLogs
Графики	Grafana
Доставка конфигурации	FluxCD
Вход по учётным записям	Keycloak

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что нужно от железа



Точные цифры на экзамене не спрашивают, но порядок величин знать надо. Платформа рассчитана на **три и более узлов** — меньше не даёт отказоустойчивости ни хранилищу, ни управляющему слою. Если узлы сами виртуальные — стенд в vSphere или Proxmox, — включите вложенную виртуализацию и проброс флагов процессора. На железе это не нужно.

Домашний стенд собирают из трёх машин по восемь ядер и двадцать четыре гигабайта — это задокументированный минимум, при котором всё работает по-настоящему.

Платформа собрана. Дальше её надо между кем-то поделить — этим и займётся следующий урок.

Что спросят на экзамене

- Порядок слоёв снизу вверх: Talos → Kubernetes → Cozystack.
- Что Talos управляется через API и не имеет SSH, а корень доступен только на чтение.
- Что платформа расширяет Kubernetes своими типами объектов, а не заменяет его.
- Названия четырёх вариантов установки (bundles) — дословно.
- Кто за что отвечает: KubeVirt, LINSTOR, SeaweedFS, Cilium, Kube-OVN, MetalLB.
- Что метрики хранит VictoriaMetrics, а не Prometheus.
- Что готовность установки проверяют по состоянию релизов FluxCD.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



2. Тенанты и доступ

Как платформа делит себя между тенантами, откуда берутся квоты и почему тенант — это не просто namespace.

Вторая по весу тема экзамена — и, пожалуй, самая полезная на практике. Если первый урок объяснял, из чего платформа собрана, то этот — как она делится между людьми.

Тенант — это объект, а не папка

В привычном Kubernetes изоляция начинается и заканчивается пространством имён. Здесь иначе: **тенант (tenant)** — это объект платформы, и создание его влечёт за собой целый набор последствий.

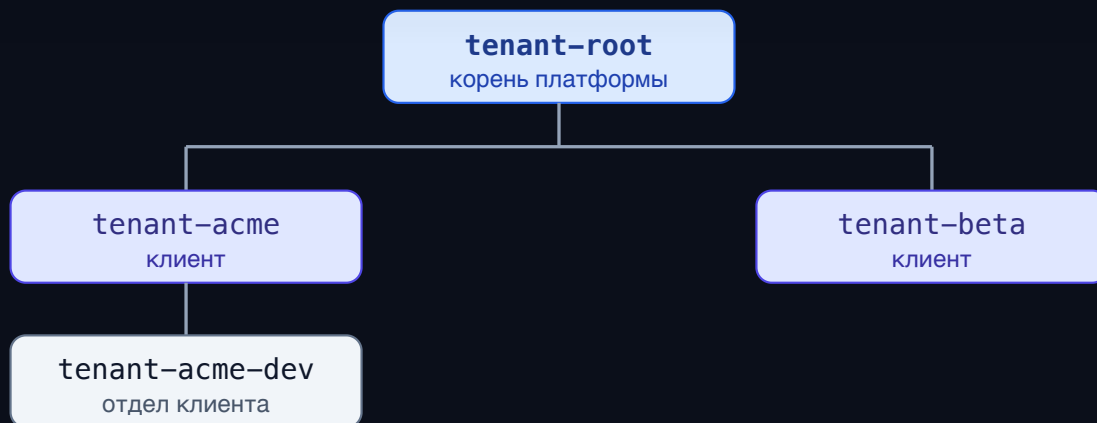
Заводите вы его так же, как всё остальное:

```
apiVersion: apps.cozystack.io/v1alpha1
kind: Tenant
metadata:
  name: асме
  namespace: tenant-root
```

Платформа в ответ создаёт пространство имён (namespace), раздаёт права, вешает сетевые политики, заводит квоты и, если попросили, поднимает внутри собственный мониторинг и хранилище.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



Тенант живёт внутри родителя. Имя пространства имён складывается из слова `tenant-` и имени.

Правило именования простое и его спрашивают: пространство имён получается приписыванием `tenant-` к имени тенанта. Тенант `асме` живёт в `tenant-асме`. Вложенный тенант создаётся **внутри пространства имён родителя**, и его собственное имя включает путь: `асме-dev`.

Зачем это нужно: провайдер отдаёт клиенту тенант, а тот делит его между своими отделами — и не приходит к провайдеру за каждым новым окружением.

Что наследуется, а что включается

У тенанта есть набор переключателей: собственный `etcd`, мониторинг, точка входа для HTTP, объектное хранилище. Каждый можно включить или оставить выключенным.

Ключевая мысль, которую любят проверять: **если сервис выключен, тенант**

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Квоты и переподписка



Тенанту назначают предел по процессорам и памяти:

```
spec:
  resourceQuotas:
    cpu: 16
    memory: 32Gi
```

А теперь то, на чём спотыкаются практически все, — **переподписка** (overcommit, `cpuAllocationRatio`, по умолчанию **10**).

Работает она так: когда вы запускаете нагрузку с пределом в четыре процессора, платформа просит у планировщика не четыре, а **предел, делённый на коэффициент** — то есть четыре десятых. Гарантируется малое, а взять при необходимости можно много.

Формула короткая, и её стоит запомнить дословно:

$$\text{запрос} = \text{предел} \div \text{cpuAllocationRatio}$$

Смысл в том, что приложения почти никогда не потребляют свой предел. Раздавать гарантии по верхней планке значит держать половину железа простаивающей. Но если про коэффициент не знать, поведение планировщика кажется бессмысленным: попросили четыре ядра, а зарезервировано меньше половины одного.

Издания

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Как люди попадают внутрь



Три пути, и это тоже спрашивают.

Дашборд — веб-интерфейс платформы. Вход через **Keycloak**, то есть по корпоративной учётной записи, а не по отдельному паролю.

kubeconfig — файл доступа, который выдаётся тенанту. Внутри него не сертификат, а вызов внешней программы: `kubectl` при первом обращении открывает браузер, вы входите через Keycloak, и выданный токен действует до истечения срока. Отсюда, кстати, требование поставить `kubectl`: без него `kubectl` не знает, как логиниться.

Terraform — тот же API, только описанием.

Права внутри тенанта раздаются обычными ролями Kubernetes. Никакой отдельной системы разрешений у платформы нет, и это ответ на популярный вопрос-ловушку.

Тенант заведён, квоты выданы, люди внутрь пущены. Теперь им надо что-то заказать.

Что спросят на экзамене

- Что тенант — объект платформы, а не просто пространство имён.
- Правило имени: пространство имён = `tenant-` плюс имя тенанта.
- Что тенанты вкладываются друг в друга, а вложенный создаётся внутри родителя.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

- Что вход идёт через Keycloak, а права — обычным RBAC.



Подробнее: [тенанты](#) · [эксплуатация платформы](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



3. Каталог управляемых сервисов

Что можно заказать у платформы, что происходит после нажатия кнопки и почему это всегда один и тот же механизм.

Каталог — то, ради чего платформу обычно и берут. Вместо «поставьте нам PostgreSQL» человек открывает список и заказывает базу, как заказывают виртуалку.

Что в списке

Категория	Позиции
Базы данных	PostgreSQL, MySQL/MariaDB, MongoDB, Redis, ClickHouse, FoundationDB
Очереди и поиск	Kafka, NATS, RabbitMQ, OpenSearch
Хранилища	бакеты S3, диски виртуальных машин
Секреты	OpenBao
Реестр образов	Harbor

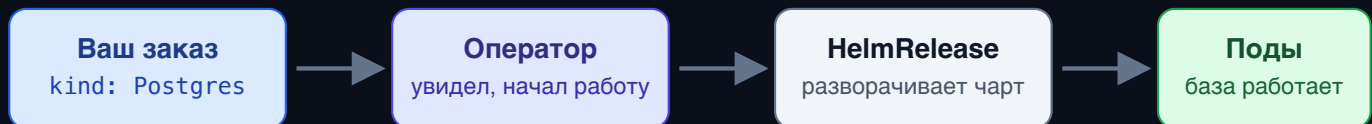
We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Один механизм на всё



Вот что стоит понять один раз и больше не возвращаться.



Один и тот же путь для базы, очереди, виртуалки и кластера.

Вы создаёте объект. Оператор его замечает и создаёт HelmRelease. Flux разворачивает чарт. Появляются поды, диски, сервисы и секреты с паролями.

Отсюда два практических вывода, которые попадают в вопросы. Первый: **дашборд не делает ничего особенного** — нажатие кнопки собирает ровно такой же объект и отправляет его в API. Второй: если сервис не поднялся, смотреть надо на HelmRelease, а не гадать по подам.

Три пути к одному и тому же

Дашборд — форма с полями. Поля берутся из описания приложения, поэтому

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что задаётся при заказе



Набор полей у каждого сервиса свой, но три вещи есть почти всегда.

Размер (`resourcesPreset`) — сколько ресурсов дать. Задаётся не числами, а готовым набором вроде `t1.micro` или `u1.medium`: платформа подставляет за ним конкретные значения.

Хранилище (`storageClass`) — сколько места и какого класса. `replicated` держит несколько копий на разных узлах, `local` быстрее, но живёт на одном.

Пользователи и базы — платформа заводит их сама. Именно поэтому в файле схемы, который вы потом накатываете, нет команд создания базы и пользователя: они уже выполнены.

Пароли платформа генерирует сама и кладёт в секрет. В дашборде он виден на вкладке `Secrets` у самого приложения.

Отказоустойчивость не бесплатна

Заказывая сервис, вы выбираете число копий. Одна копия — учебный стенд: узел ушёл, сервис ушёл с ним. Две и больше — платформа сама следит, кто главный, и переключает при отказе.

Здесь же живёт частая ловушка: **отказоустойчивость сервиса и сохранность данных — разные вещи**. Копии спасают от падения узла, но не от удалённой

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



- Что заказ любого сервиса — это объект в API, а дашборд лишь собирает его за вас.
- Цепочку: объект → оператор → HelmRelease → работающие поды.
- Что диагностику начинают с состояния HelmRelease.
- Что пользователей и базы заводит платформа, а пароли кладёт в секрет.
- Разницу между `replicated` и `local`.
- Что число копий защищает от отказа узла, но не заменяет резервные копии.

Подробнее: [управляемые приложения](#) · [кластеры Kubernetes](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



4. Виртуальные машины

Как виртуалка становится обычной нагрузкой Kubernetes, зачем диск отдельно от машины и где проходят честные границы.

Тема, на которой администратору VMware проще всего — и на которой он же чаще всего попадает, потому что привычные аналогии тут работают лишь наполовину.

Машина как обычная нагрузка

Виртуализацию обеспечивает **KubeVirt**. Идея у него такая: виртуальная машина — это процесс, который запускается в поде, а управляется как всё остальное в кластере.

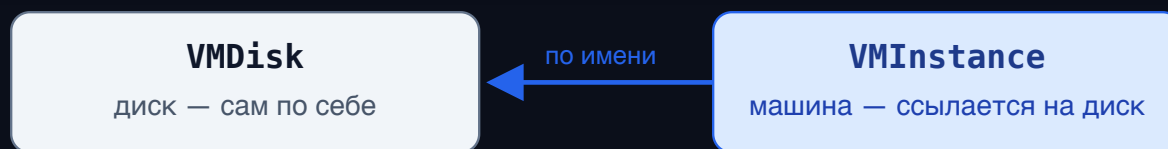
Из этого следует то, что для человека из мира vSphere звучит непривычно. Планировщик Kubernetes выбирает узел для машины так же, как для приложения. Ресурсы считаются теми же запросами и пределами. Права выдаются тем же RBAC.

Гипервизор при этом честный — KVM, тот же, что в любом Linux.

Два объекта вместо одного

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



Диск переживает машину: удалили машину — диск остался.

Его можно подключить к другой или удалить отдельно.

В vSphere диск — свойство машины. Здесь — самостоятельный объект.

Диск создаётся отдельно и живёт своей жизнью. Содержимое он берёт одним из четырёх способов, и их стоит различать: клон готового образа из каталога платформы, загрузка по ссылке HTTP, копия другого диска, выгрузка файла со своей машины.

Как попасть внутрь

Здесь заканчиваются привычки и начинается специфика.

cloud-init — стандартный механизм облачных образов: при первом запуске система читает описание и настраивает себя. Пароль, ключи, пакеты, команды. Это ближайший аналог Customization Specification, только описанный текстом рядом с самой машиной.

virtctl — отдельная утилита для консоли, экрана и проброса портов.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Что умеет и чего не умеет

Экзамен любит вопросы про возможности — и здесь важно не перепутать, чего платформа не делает, с тем, что она делает иначе.

Живая миграция есть. Работающая машина переезжает на другой узел без остановки — этим пользуются, когда узел нужно вывести на обслуживание.

Снимки состояния есть. Их обеспечивает слой хранилища.

Проброс видеокарт есть, причём настраивается автоматически. Список устройств, которые разрешено пробрасывать, задаёт платформа, а не тенант.

Чего действительно нет — **автоматической балансировки нагрузки между узлами.** В vSphere этим занимается DRS: он сам решает, что машину пора перевезти, и везёт. Здесь такого нет. Планирование ёмкости и перевоз машин во время обслуживания делаются осознанно, а не сами собой.

Разница тонкая, но именно на ней строят вопросы: миграция как **инструмент** есть, миграция как **постоянно работающая автоматика** — нет.

Машина работает. Пакеты до неё довозит сеть — а там четыре компонента, которые легко перепутать между собой.

Что спросят на экзамене

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

- Что `virtctl` требует префикса `vmi/`, и почему — из-за прав на подресурс.
- Что живая миграция, снимки состояния и проброс видеокарт есть.
- Что автоматической балансировки между узлами, как у DRS, нет.



Подробнее: [виртуализация](#) · [хранилище](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



5. Сети

Четыре компонента, каждый со своей задачей: кто носит пакеты внутри, кто делит сети между тенантами, кто выдаёт внешние адреса и кто пускает HTTP.

Сетевая часть выглядит сложной, пока не разложить её по задачам. Компонентов четыре, и у каждого своя работа — экзамен спрашивает именно распределение ролей, а не настройки.

Ingress / Gateway

пускает HTTP снаружи внутрь, отдельно у каждого тенанта

MetalLB

выдаёт внешние адреса на своём железе, без облачного балансировщика

Kube-OVN

отдельные сети тенантов, VPC, выдача адресов

Cilium

носит пакеты между подами, применяет сетевые политики

Снизу вверх: от пакетов между подами до входа снаружи.

Cilium — основа

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Именно Cilium исполняет ту изоляцию между тенантами, о которой шла речь во втором уроке.



Разделение труда между ним и Kube-OVN стоит запомнить, потому что оба называются «сетью»: **Cilium** — **основной CNI**, он даёт сеть подам и применяет политики для всех. **Kube-OVN работает поверх** и нужен там, где требуются отдельные сети со своими диапазонами: сети виртуальных машин и VPC тенантов. На одном поде они не пересекаются.

Kube-OVN — сети тенантов

Cilium даёт одну плоскую сеть на кластер. Этого мало, когда тенантов много и каждому нужна своя, с собственными диапазонами адресов, — возможно, пересекающимися с чужими.

Этим занимается Kube-OVN: отдельные виртуальные сети, свои диапазоны, выдача адресов, маршрутизация между сетями там, где она разрешена.

Ближайшая аналогия из привычного мира — VPC у облачных провайдеров или сети NSX.

MetalLB — внешние адреса

В публичном облаке сервис типа LoadBalancer получает адрес от провайдера. На своём железе провайдера нет, и без MetalLB такой сервис вечно висит в

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Для веб-приложений выдавать каждому свой адрес расточительно. **Ingress** принимает запросы на общий адрес и раскладывает их по именам и путям.

Особенность платформы: **точка входа (ingress) своя у каждого тенанта**. Это не общий контроллер на весь кластер — тенант с включённым ingress получает собственный, со своими правилами и своими сертификатами. Тенант без него пользуется родительским, как и с остальными сервисами.

Имена и сертификаты

Два помощника, которые обычно не замечают, пока они работают.

CoreDNS отвечает за имена внутри кластера. Именно поэтому приложения обращаются друг к другу по именам вида `postgres-db-rw`, а не по адресам: имя переживает переезд базы на другой узел, а адрес — нет.

cert-manager выпускает сертификаты и продлевает их сам, без напоминаний.

Всё это работает ровно до первого сбоя. О том, как о сбое узнать и что стоит подготовить заранее, — следующий урок.

Что спросят на экзамене

- Что Cilium носит пакеты между подами и работает на eBPF.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

- Почему в настройках приложений пишут имена сервисов, а не адреса.



Подробнее: [сети платформы](#)

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.



6. Наблюдаемость и резервные копии

Кто собирает метрики и журналы, где их смотреть и чем резервная копия отличается от отказоустойчивости.

Две темы в одном уроке, потому что обе про одно: что делать, когда что-то пошло не так, — и что вы успели подготовить заранее.

Метрики и журналы

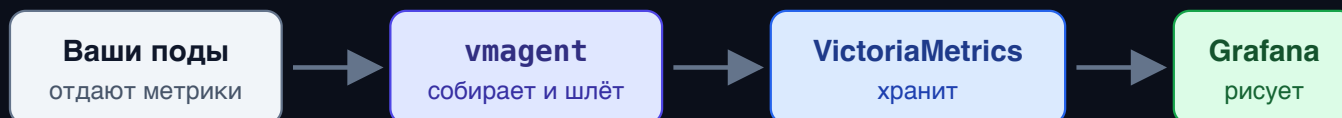
Стек здесь не тот, который ожидают по привычке, и это любимая ловушка экзамена.

Что собирают	Чем
Метрики	VictoriaMetrics
Журналы	VictoriaLogs
Графики	Grafana
Оповещения	Alerta

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Alerta сводит срабатывания правил в одно место, чтобы одна авария не приходила восемью письмами из восьми источников.



Тот же путь у журналов, только вместо vmagent — fluent-bit, а вместо VictoriaMetrics — VictoriaLogs.

Про наследование помните из второго урока: тенант без своего мониторинга шлёт метрики родительскому. А вот включать сбор **задним числом бесполезно** — записей за прошлое взять неоткуда, и это стоит решать при создании кластера, а не когда график понадобился.

Резервные копии

Здесь два разных механизма, и путать их — верный способ ошибиться в вопросе.

Копии управляемых сервисов (backup). Заказывая базу, вы включаете резервное копирование прямо в её настройках. Дальше платформа снимает копии по расписанию и складывает их в объектное хранилище. Отвечают за это три вещи: **BackupClass** описывает, куда и как складывать, **расписание** — когда, а **BackupJob**

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Где проходит граница



Самое ценное в этой теме — понимать, чего резервные копии **не** делают.

Копии — не отказоустойчивость. Реплики спасают от падения узла, копии — от удалённых данных. Это разные беды, и одно другое не заменяет.

Копия, которую ни разу не разворачивали, — не копия, а надежда. Восстановление надо пробовать, пока оно не понадобилось.

И главное: копия хранится в объектном хранилище. Если оно живёт в том же кластере, что и данные, то от потери кластера целиком она не спасёт. Для настоящей защиты хранилище должно быть снаружи.

Мы разобрали, из чего платформа сделана и что она умеет. Остался последний вопрос — почему она устроена именно так.

Что спросят на экзамене

- Что метрики хранит VictoriaMetrics, а журналы — VictoriaLogs. Не Prometheus и не Loki.
- Что VictoriaMetrics совместима с PromQL.
- Что метрики собирает vmagent.
- Что сбор метрик нельзя включить задним числом.
- Три составляющие резервного копирования сервисов: BackupClass, расписание, BackupJob.
- Что Velero работает на уровне платформы, а не отдельного сервиса.
- Что реплики и резервные копии решают разные задачи.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

7. Как это вообще работает

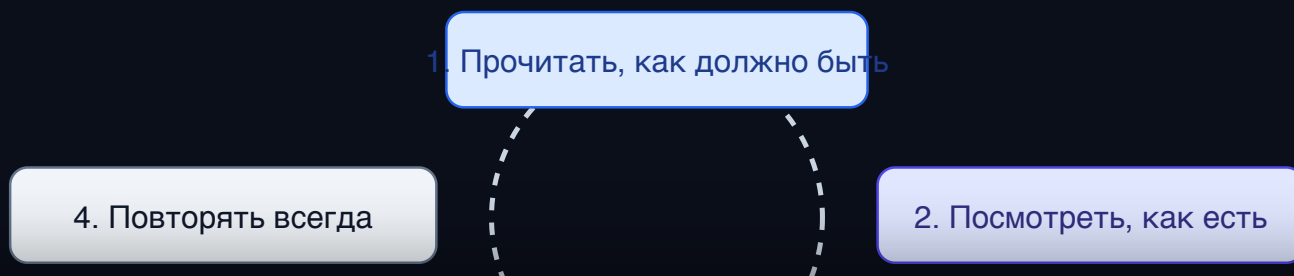
Желаемое состояние, операторы и GitOps — четыре идеи, на которых держится всё остальное.

Последний урок — про идеи, а не про компоненты. Они лежат в основании всего, о чём шла речь раньше, и вопросы по ним формулируются просто: «почему это работает именно так».

Вы описываете результат, а не действия

Привычный подход к администрированию — последовательность шагов: поставь, настрой, запусти, проверь. Здесь иначе: вы описываете, **как должно быть** — желаемое состояние, *desired state*, а система сама решает, что для этого сделать.

Разница проявляется, когда что-то ломается. Скрипт, отработавший вчера, сегодня не поможет: он уже выполнен. А описание желаемого состояния действует постоянно — если реальность от него отклонилась, её вернут обратно.



We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Новые типы объектов и операторы



`kubectl get tenants` работает, хотя в Kubernetes никаких тенантов нет. Дело в **CRD** (custom resource definition) — механизме, которым в API добавляют собственные типы.

Но тип сам по себе ничего не делает: это запись, которую кластер согласен хранить. Работу выполняет **оператор** (operator) — программа, которая следит за объектами своего типа и приводит мир в соответствие.

Отсюда практический вывод: если объект создан, а ничего не произошло, вопрос не к объекту, а к оператору. Он либо не запущен, либо не смог.

GitOps

Раз состояние описывается текстом, текст можно хранить в системе контроля версий. Тогда Git становится единственным источником истины, а специальная программа следит, чтобы кластер ему соответствовал.

Платформа использует **FluxCD** и применяет этот подход **к самой себе**: её компоненты описаны и разворачиваются тем же способом, каким вы разворачивали бы своё приложение.

Что это даёт на практике: у любого изменения есть автор и время, откат — это возврат к прошлому состоянию файла, а правка, сделанная руками в обход Git, будет отменена при следующей свертке. Последнее раздражает ровно до первого

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.

Один объект — это один объект. Приложение — это обычно десяток: развёртывание, сервис, настройки, секреты, правила доступа.

Helm упаковывает такой набор в **чарт** — шаблоны плюс значения. Меняя значения, вы получаете разные установки из одной упаковки.

В платформе Helm — не рекомендация, а несущая конструкция: каждая позиция каталога это чарт, и заказ сервиса разворачивает именно его.

Складывается такая цепочка, и она стоит того, чтобы её запомнить целиком:

объект → оператор → HelmRelease → чарт → работающие поды

Всё, что делает платформа, проходит по ней. Заказали базу — прошло по ней. Создали тенант — прошло по ней. Установилась сама платформа — тоже.

Что спросят на экзамене

- Что описывается желаемое состояние, а не последовательность действий.
- Что контроллер непрерывно сверяет желаемое с фактическим и устраняет разницу.
- Что CRD добавляет тип, а работу выполняет оператор.
- Что платформа применяет GitOps к самой себе через FluxCD.
- Что ручная правка в обход Git будет отменена при следующей сверке.
- Что чарт — единица упаковки, и каждая позиция каталога это чарт.
- Цепочку: объект → оператор → HelmRelease → чарт → поды.

We use cookies and tracking technologies

We use essential cookies to make this website work. With your consent, we also use analytics, visitor identification, enrichment, and sales/marketing automation tools to understand how visitors use our site, identify company-level interest, improve our content, and support relevant B2B communications. You can accept all cookies, reject non-essential cookies, or manage your preferences at any time.